SCIENCE CORPS — PHILIPPINE MANAGER'S OFFICE

Fellowship Scanner Implementation Guide

Step-by-step workflow for the email blast analytics pipeline

Prepared for: Prince Nayga | June 28, 2026

CONTENTS

1 — What This System Does

After you send a fellowship blast email to your academic listserv, this system automatically:

- Connects to your Gmail and finds every reply (inbox, updates, spam, and bounce folders)
- Auto-tags hard bounces (mailer-daemon, postmaster) without any manual step
- Tracks which listserv addresses never replied at all (No Reply rows)
- Exports unclassified emails to `output/pending_for_claude.xlsx` for upload to Claude.ai
- Produces a colour-coded Excel report with an Action Plan tab telling you exactly what to do
- Generates a draft outreach email for every positive reply
- Outputs a cleaned listserv (dead bounces removed) ready for the next blast

> **Three scripts, three outputs**
>
> `fellowship_scanner.py` → main Excel report | `positive_replies_report.py` → outreach HTML/PDF | `clean_listserv.py` → cleaned listserv Excel

2 — One-Time Setup

Do this once on each machine you use. Skip any step you have already completed.

1 Get the project files

Clone or copy the project folder to your Desktop (or any folder you prefer).

The folder should contain: `fellowship_scanner.py`, `clean_listserv.py`, `positive_replies_report.py`, `requirements.txt`

2 Create and activate a Python virtual environment

Open a terminal in the project folder and run:

```
python -m venv venv
venv\Scripts\activate
```

Your prompt will show `(venv)` when the environment is active. Always activate it before running any script.

3 Install dependencies

```
pip install -r requirements.txt
```

This installs the Google API client and openpyxl (Excel). Do this once; no need to repeat unless you reset the venv.

4 Get your Google OAuth credentials

1. Go to **console.cloud.google.com** and sign in with `pnayga@science-corps.org`
2. Select your project (or create one named *Fellowship Scanner*)
3. Go to **APIs & Services** → **Library** → enable **Gmail API**
4. Go to **APIs & Services** → **Credentials** → click **Create Credentials** → **OAuth client ID**
5. Choose **Desktop App**, name it anything, click **Create**
6. Click the download icon → save the file as `credentials.json` in the project folder

> **Never share or commit credentials.json**
> This file is your private Google login key. Keep it only on your machine.

5 First run — browser login

The first time you run the scanner, a browser window will open asking you to sign in to Gmail. After you approve, a `token.json` file is saved. Future runs skip the browser login automatically.

> **token.json is private — do not share or commit it.**
>
> If it is ever lost, just delete it and the next run will prompt you to log in again.

3 — Before Every Blast — Prepare Your Listserv

1 Place your listserv file in the project folder

Your listserv should be an Excel file (`.xlsx`) with one email address per row (no header needed). Place it in the project folder.

The current file is: `Filtered Listserv for Feb 2026 Applications.xlsx`

For a new cycle, either update this file or change `LISTSERV_CSV` in the CONFIG block of `fellowship_scanner.py` to point to the new filename.

2 Update the CONFIG block in fellowship_scanner.py

Open `fellowship_scanner.py` in any text editor and update these values near the top:

Setting	What to change
`SEARCH_AFTER_DATE`	Set to the date you sent the blast (YYYY/MM/DD). Emails before this date are ignored.
`LISTSERV_CSV`	Filename of your current listserv Excel file.
`CC_EMAILS`	Add any addresses you CC'd on the blast — prevents them from appearing as false positives.

3 Send the blast (BCC all listserv addresses)

Send your fellowship email using BCC for all listserv addresses. Wait at least 24–48 hours before running the scanner so bounces and auto-replies have time to arrive.

4 — Running the Scanner After a Blast

1 Open a terminal in the project folder and activate the venv

```
venv\Scripts\activate
```

2 Run the scanner

```
python fellowship_scanner.py
```

The script will:

- Search Gmail across all folders (inbox, updates, spam, bounces)
- Run a batch search for any reply from your listserv addresses
- Auto-tag hard bounces (mailer-daemon / postmaster) immediately
- Add No Reply rows for every listserv address that didn't respond
- Save `output/fellowship_replies.xlsx` with bounces and No Reply rows filled in
- Save `output/pending_for_claude.xlsx` with the remaining unclassified emails

Runtime is typically **10–20 minutes** for 800+ listserv addresses (Gmail batch queries take time).

3 Re-running is safe

You can run the scanner multiple times. It skips rows already in the Excel file (matched by email address + date) and only adds new entries. The Action Plan and Summary tabs are always rebuilt fresh.

> **Tip:**
> If you delete `output/fellowship_replies.xlsx` before re-running, everything is rebuilt from scratch — useful if the file gets corrupted or you want a clean slate.

5 — Classifying Pending Emails via Claude.ai

After the scanner runs, some emails cannot be auto-tagged and are saved to `output/pending_for_claude.xlsx`. These need to be classified using the Science Corps Classification project in Claude.ai.

1 Open the Science Corps Classification project in Claude.ai

Go to **claude.ai** and open the **Science Corps Classification** project. Start a new conversation.

2 Upload pending_for_claude.xlsx

Click the attachment / upload icon in the Claude.ai chat and upload:

`output/pending_for_claude.xlsx`

Claude will read each email row, classify it into a category, and generate a completed `fellowship_replies.xlsx` with all rows colour-coded and the Action Plan filled in.

3 Download and save the completed report

When Claude finishes, download the file it provides and save it to:

`output/fellowship_replies.xlsx`

(Replace the existing file.)

What if there are no pending emails?

If all emails were auto-tagged as bounces and the rest are No Reply, the script will print "No pending emails" and skip generating the pending file. In that case, your `fellowship_replies.xlsx` is already complete.

6 — Reading the Excel Report

The report is saved at `output/fellowship_replies.xlsx` and has three sheets:

Sheet 1 — Action Plan (open this first)

A prioritised to-do list rebuilt every run. See Part 7 for details.

Sheet 2 — Summary

Total counts by classification category and Gmail category. Use this for a quick overview.

Sheet 3 — Replies

Every email and listserv address, one row each, colour-coded:

Positive / Interested | **Has a Question** | **Application Submitted** | **No Longer Works There** | **Bounce / Delivery Failure** | **Declined / Not Interested** | **Auto-Reply (OOO)** | **No Reply** | **Pending (unclassified)**

> **Important — No Reply count**
>
> ~700 No Reply rows is normal and expected. The listserv has 800+ addresses; a 10–15% response rate (including bounces and auto-replies) is typical for a cold academic blast. The No Reply rows are not a data error.

7 — The Action Plan — What to Do Next

Open the **Action Plan** tab in the Excel report and work through each step in order.

1 STEP 1 — FOLLOW UP NOW *(Positive, Has a Question, Application Submitted)*

- **Positive / Interested:** Reply with the application link, deadlines, and next steps. Then run `positive_replies_report.py` (see Part 8) to get a personalised outreach email asking them to refer colleagues.
- **Has a Question:** Read the summary and reply directly to their question.
- **Application Submitted:** Acknowledge receipt and add them to your applicant tracker.

2 STEP 2 — SPECIAL ACTION NEEDED *(No Longer Works There, Declined)*

- **No Longer Works There:** Read the summary — they may have provided a forwarding contact or new colleague's email. If so, add that new address to your listserv for next cycle and remove the old one.
- **Declined / Not Interested:** Read the summary carefully.
  - If they said "*post to our board / listserv*" → post the opportunity there as instructed.
  - If they said "*don't use BCC, send directly*" → resend a direct individual email.
  - If they gave a plain no → acknowledge and remove from future blasts.

3 STEP 3 — INCLUDE IN NEXT BLAST *(No Reply, Auto-Reply)*

No action needed now. These addresses are valid — people either did not see the email or were out of office. They will be included in the cleaned listserv for the next cycle automatically.

4 STEP 4 — DEAD ADDRESSES — REMOVE FROM LIST *(Bounce)*

These email addresses failed to deliver. Run `clean_listserv.py` (Part 9) to strip them from your list. No other action needed.

8 — Generating the Positive Replies Outreach

1 Run the script

```
python positive_replies_report.py
```

This reads `output/fellowship_replies.xlsx` and generates `output/positive_replies_outreach.html`.

2 Convert to PDF

1. Open `output/positive_replies_outreach.html` in Chrome or Edge
2. Press **Ctrl + P**
3. Change destination to **Save as PDF**
4. Click **Save**

3 Review and personalise before sending

Each card in the report shows:

- The sender's name, email, institution, and date
- A summary of what they said in their reply
- A ready-to-send draft email — personalised by first name — thanking them and asking if they know any STEM PhD graduates who might be interested

> **Tip:**
> The draft is a starting point. Adjust the tone for each person if you have extra context from their reply.

9 — Cleaning the Listserv for the Next Cycle

1 Run the cleaner

```
python clean_listserv.py
```

This generates `output/cleaned_listserv.xlsx` with three sheets:

Sheet	Contents	What to do with it
`Cleaned Listserv`	Addresses with confirmed dead bounces removed	Use this as your listserv for the next blast
`Needs Review`	No Longer Works There + Declined rows with summaries	Read each row and decide: remove, update, or follow up
`Removed (Bounces)`	Confirmed dead addresses	Keep for your records; do not re-add to any list

2 Manually process the Needs Review sheet

- For **No Longer Works There** — check if they gave a new contact. If yes, add the new address to the Cleaned Listserv sheet.
- For **Declined** — if they said "don't BCC us", remove them. If they said "post to our board", that is a one-time action already done.

3 Save the cleaned list for the next cycle

Save `output/cleaned_listserv.xlsx` in the project folder under a new name (e.g. `Listserv July 2026.xlsx`) and update `LISTSERV_CSV` in the CONFIG block of `fellowship_scanner.py` before the next blast.

10 — Full File Reference

File	Purpose	Edit / commit?
`fellowship_scanner.py`	Main scanner — Gmail fetch, auto-tag, Excel export, pending file	Edit CONFIG block only; commit freely
`clean_listserv.py`	Generates cleaned listserv Excel from the report	No edits needed; commit freely
`positive_replies_report.py`	Generates outreach HTML for positive replies	No edits needed; commit freely
`generate_guide.py`	Generates this implementation guide HTML	No edits needed; commit freely
`requirements.txt`	Python package list	No edits needed; commit freely
`credentials.json`	Google OAuth2 key	**NEVER commit.** Keep locally only.
`token.json`	Saved Gmail session	**NEVER commit.** Delete to force re-login.
`Listserv *.xlsx`	Your current mailing list (one email per row)	Update each cycle; keep out of public repos.
`output/fellowship_replies.xlsx`	Main colour-coded report (Action Plan, Summary, Replies)	Auto-generated; do not manually edit.
`output/pending_for_claude.xlsx`	Unclassified emails — upload to Claude.ai for classification	Auto-generated; upload to Claude.ai then discard.
`output/cleaned_listserv.xlsx`	Filtered listserv for next blast	Auto-generated; manually adjust Needs Review sheet.
`output/positive_replies_outreach.html`	Outreach drafts for positive replies	Auto-generated; print to PDF in Chrome.

11 — Quick-Reference Checklist

Print this page and check off each item every cycle.

One-time setup (first machine only)

- ☐ Create Python virtual environment and install requirements.txt
- ☐ Download credentials.json from Google Cloud Console
- ☐ Run scanner once to trigger Gmail browser login (token.json saved)

Before every blast

- ☐ Update `SEARCH_AFTER_DATE` in fellowship_scanner.py to the blast send date
- ☐ Update `LISTSERV_CSV` to point to the current listserv Excel file
- ☐ Confirm `CC_EMAILS` includes anyone you will CC (not BCC)
- ☐ Send blast via BCC
- ☐ Wait 24–48 hours for replies and bounces to arrive

After every blast (run in this order)

- ☐ `python fellowship_scanner.py` — fetches replies, auto-tags bounces, adds No Reply rows, saves Excel + pending file
- ☐ Upload `output/pending_for_claude.xlsx` to the Science Corps Classification project in Claude.ai
- ☐ Download completed `fellowship_replies.xlsx` from Claude.ai → save to `output/`
- ☐ Open **Action Plan** tab — work through Step 1 (follow ups) and Step 2 (special cases)
- ☐ `python positive_replies_report.py` — generate outreach HTML, print to PDF
- ☐ Send personalised thank-you + referral request to each positive reply
- ☐ `python clean_listserv.py` — generate cleaned listserv
- ☐ Review Needs Review sheet, adjust cleaned list manually if needed
- ☐ Save final cleaned list as new Excel file for next cycle

> **Tip — Re-running the scanner**
> You can run `fellowship_scanner.py` multiple times safely. New replies are appended; existing rows are never duplicated. Run it again after a few days to catch late replies.
